

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
Khoa Điện - Điện tử



BÁO CÁO HỌC PHẦN MỞ RỘNG
KỸ THUẬT SỐ - EE100D
ĐỀ TÀI:
THIẾT KẾ RTL

Giảng viên hướng dẫn: Phan Võ Kim Anh

Nhóm: 8

STT	Thành viên nhóm	MSSV	% Đóng góp
1	Lê Nhật Nam	2412169	100%
2	Lê Tấn Nhật Tân	2414122	100%
3	Nguyễn Anh Đạt	2410686	100%
4	Lê Nguyên Hưng	2411346	100%

TP. Hồ Chí Minh, tháng 12/2025

Mục lục

1	Mở đầu	2
1.1	Lời cảm ơn	2
1.2	Tóm tắt đề tài	3
1.2.1	Lab 4: A Simple Processor	3
1.2.2	Lab 5: An Enhanced Processor	3
2	A SIMPLE PROCESSOR	4
2.1	Thí nghiệm 1(Design and implement a simple processor)	4
2.1.1	Code:	4
2.1.2	Mô phỏng sóng	8
2.1.3	RTL Viewer	9
2.2	Thí nghiệm 2: Processor với ROM và Counter	9
2.2.1	Counter và đọc file mif (memory.sv)	9
2.2.2	Processor cập nhật cho Thí nghiệm 2 (proc.sv)	10
2.2.3	Mô phỏng sóng	14
2.2.4	RTL Viewer	14
3	AN ENHANCED PROCESSOR	15
3.1	Yêu cầu bài toán	15
3.2	Thiết kế chi tiết	15
3.2.1	Các module chức năng (datapath.sv)	15
3.2.2	module Datapath (datapath.sv)	16
3.2.3	Module proc (proc.sv) - Control Unit	17
3.2.4	Module Memory và Bộ nhớ	20
3.2.5	Module System Top	21
3.2.6	Module Top Level (FPGA Wrapper)	22
3.3	Kết quả mô phỏng	23
3.4	Sơ đồ RTL Viewer	24
4	Lời kết	26

1 Mở đầu

1.1 Lời cảm ơn

Lời đầu tiên, nhóm chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến cô **Phan Võ Kim Anh**. Cô không chỉ truyền đạt những kiến thức nền tảng quý báu về Kỹ thuật số mà còn tận tình hướng dẫn, định hướng giải pháp và hỗ trợ kỹ thuật trong suốt quá trình nhóm thực hiện đề tài. Những ý kiến đóng góp của cô là kim chỉ nam giúp chúng em hoàn thiện mô hình và báo cáo này.

Chúng em cũng xin gửi lời cảm ơn đến các anh chị khóa trên và bạn bè đã nhiệt tình chia sẻ tài liệu tham khảo, kinh nghiệm thiết kế mạch và hỗ trợ xử lý các vấn đề phát sinh trong quá trình thi công thực tế.

Dù đã nỗ lực hết mình, nhưng do giới hạn về thời gian và kiến thức, báo cáo khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp quý báu từ cô và các bạn để đề tài được hoàn thiện hơn.

TP. Hồ Chí Minh, ngày 30 tháng 12 năm 2025

Nhóm thực hiện đề tài

1.2 Tóm tắt đề tài

1.2.1 Lab 4: A Simple Processor

Mục tiêu: Lắp ráp các khối phần cứng đã thiết kế ở Pre-Lab thành một bộ xử lý (CPU) hoàn chỉnh và mô phỏng hoạt động.

1. **Experiment 1: Lắp ráp CPU:** CPU được xây dựng bằng cách kết nối các khối: hệ thống thanh ghi, bộ cộng trừ (Add/Sub) và khối điều khiển FSM bằng **SystemVerilog**. Một module **top-level** được tạo để quản lý luồng dữ liệu và tín hiệu điều khiển DIN, Run, Reset_n trong quá trình thực thi lệnh.
2. **Experiment 2: Tự động hóa nạp lệnh:** Processor được kết nối với bộ nhớ **ROM** 32×9 và một bộ đếm địa chỉ 5-bit. Counter tự động tăng để đọc lệnh từ ROM và nạp trực tiếp vào Processor, thay thế việc nạp lệnh thủ công bằng công tắc.

1.2.2 Lab 5: An Enhanced Processor

Mục tiêu: Nâng cấp CPU để giao tiếp với bộ nhớ và thiết bị ngoại vi, đồng thời triển khai trên kit FPGA thực tế.

1. **Nâng cấp kiến trúc:** Processor được bổ sung các cổng giao tiếp ADDR, DOUT và W. Hệ thống được kết nối với RAM (đọc/ghi dữ liệu) và một Output Register dùng để điều khiển đèn LED.
2. **Memory Mapping:** Các bit địa chỉ cao A_7, A_8 được sử dụng để phân biệt vùng địa chỉ: RAM hoặc Output Register. Cơ chế này cho phép CPU ghi dữ liệu vào bộ nhớ hoặc xuất kết quả trực tiếp ra LED.
3. **Thực hành trên FPGA:** Chương trình điều khiển CPU được viết dưới dạng file **.mif**, thực hiện vòng lặp và hiển thị giá trị đếm lên LED. Hệ thống được nạp lên board FPGA dòng **DE-series** và hoạt động ở xung nhịp **50 MHz**.

2 A SIMPLE PROCESSOR

2.1 Thí nghiệm 1(Design and implement a simple processor)

Yêu cầu: Thiết kế và triển khai một bộ xử lý đơn giản (Simple Processor) có độ rộng dữ liệu 9-bit trên chip FPGA. Thí nghiệm tập trung vào việc hiện thực hóa kiến trúc cơ bản của một bộ xử lý 9-bit bao gồm các thanh ghi R0-R7, bộ cộng/trừ (AddSub), bộ dồn kênh (Mux) và khối điều khiển FSM. Chức năng chính là thực thi 4 lệnh: di chuyển dữ liệu (mv, mvi) và tính toán số học (add, sub) thông qua việc điều phối dữ liệu trên hệ thống Bus.

2.1.1 Code:

Module thanh ghi (regn): Module này thực hiện chức năng lưu trữ dữ liệu 9-bit đồng bộ theo cạnh lên của xung nhịp Clock. Khi tín hiệu cho phép nạp Rin bằng 1, dữ liệu đầu vào R sẽ được nạp vào thanh ghi; ngược lại, dữ liệu cũ sẽ được duy trì tại ngõ ra Q.

```
1 module regn #(parameter n = 9) (  
2     input  logic [n-1:0] R,  
3     input  logic Rin, Clock,  
4     output logic [n-1:0] Q  
5 );  
6     always_ff @(posedge Clock) begin  
7         if (Rin) Q <= R;  
8     end  
9 endmodule
```

Listing 1: Module regn (Register N-bit)

Module ALU (AddSub) Module này thực hiện chức năng cộng hoặc trừ hai số 9-bit dựa trên mạch logic tổ hợp. Khi tín hiệu điều khiển Sub bằng 1, kết quả xuất ra là hiệu $A - B$; ngược lại, khi Sub bằng 0, kết quả là tổng $A + B$.

```
1 module alu (  
2     input  logic [8:0] A, B,  
3     input  logic Sub,  
4     output logic [8:0] S  
5 );  
6     always_comb begin  
7         if (Sub) S = A - B;  
8         else    S = A + B;  
9     end  
10 endmodule
```

Listing 2: Module alu (Arithmetic Logic Unit)

Module Decoder (dec3to8) Module này đóng vai trò quan trọng trong việc chọn thanh ghi đích hoặc nguồn dựa trên đầu vào 3-bit, xuất ra tín hiệu 8-bit Y. Khi tín hiệu cho phép En bằng 1, một trong tám ngõ ra của Y sẽ được kích hoạt tương ứng với giá trị nhị phân của đầu vào W; ngược lại, nếu En bằng 0, tất cả các ngõ ra đều bằng 0

```
1 module dec3to8 (  
2     input  logic [2:0] W,  
3     input  logic En,
```

```

4      output logic [7:0] Y
5  );
6      always_comb begin
7          if (!En) Y = 8'b0;
8          else begin
9              case (W)
10                 3'b000: Y = 8'b0000_0001; // R0
11                 3'b001: Y = 8'b0000_0010; // R1
12                 3'b010: Y = 8'b0000_0100; // R2
13                 3'b011: Y = 8'b0000_1000; // R3
14                 3'b100: Y = 8'b0001_0000; // R4
15                 3'b101: Y = 8'b0010_0000; // R5
16                 3'b110: Y = 8'b0100_0000; // R6
17                 3'b111: Y = 8'b1000_0000; // R7
18                 default: Y = 8'b0;
19             endcase
20         end
21     end
22 endmodule

```

Listing 3: Module dec3to8 (Decoder 3-to-8)

Đây là module trung tâm kết nối tất cả các thành phần để tạo thành một bộ xử lý hoàn chỉnh. Hệ thống sử dụng một máy trạng thái hữu hạn (FSM) gồm 4 bước (từ T0 đến T3) để điều phối dữ liệu giữa các thanh ghi thông qua một hệ thống Bus dùng chung. Chức năng cụ thể bao gồm:

- **Giải mã lệnh:** FSM nhận mã máy từ thanh ghi lệnh (IR) để xác định loại thao tác cần thực hiện (mv, mvi, add, hoặc sub).
- **Điều phối luồng dữ liệu:** FSM kích hoạt các tín hiệu điều khiển để xác định dữ liệu nào được phép đưa lên Bus (thông qua các tín hiệu như Rout, Gout, DINout) và dữ liệu trên Bus sẽ được nạp vào thanh ghi nào (thông qua Rin, Ain, Gin, IRin).
- **Kiểm soát ALU:** Điều khiển bộ cộng/trừ thực hiện đúng phép tính thông qua tín hiệu AddSub khi xử lý lệnh cộng hoặc trừ.
- **Báo hiệu trạng thái:** Phát tín hiệu Done khi lệnh đã hoàn thành để hệ thống chuẩn bị cho lệnh tiếp theo.
- **Quản lý các bước thời gian:** FSM chia quá trình thực thi lệnh thành các bước thời gian (T0, T1, T2, T3):
 - **T0:** Luôn thực hiện nạp lệnh từ cổng DIN vào thanh ghi IR.
 - **T1 đến T3:** Thực hiện các bước luân chuyển dữ liệu cụ thể tùy theo độ phức tạp của lệnh (ví dụ: lệnh add cần nhiều bước để nạp toán hạng vào thanh ghi đệm trước khi tính toán).

```

1 module simpleCPU (
2     input  logic [8:0] DIN,
3     input  logic      Resetn,
4     input  logic      Clock,
5     input  logic      Run,
6     output logic      Done,
7     output logic [8:0] BusWires,
8     output logic [1:0] Tstep_View,

```

```
9      output logic [8:0] R0_Q, R1_Q, R2_Q, R3_Q,
10     output logic [8:0] R4_Q, R5_Q, R6_Q, R7_Q,
11     output logic [8:0] A_Q, G_Q,
12     output logic [8:0] IR_Q
13 );
14
15     logic [7:0] Rin_HotOne, Rout_HotOne;
16     logic [7:0] Xreg, Yreg;
17     logic IRin, DINout, Ain, Gin, Gout, AddSub;
18     logic [8:0] Alu_Result;
19
20     typedef enum logic [1:0] {T0=2'b00, T1=2'b01, T2=2'b10, T3=2'b11} state_t;
21     state_t Tstep_Q, Tstep_D;
22
23     logic [2:0] I, X, Y;
24
25     assign Tstep_View = Tstep_Q;
26
27     assign I = IR_Q[8:6];
28     assign X = IR_Q[5:3];
29     assign Y = IR_Q[2:0];
30
31     dec3to8 decX (.W(X), .En(1'b1), .Y(Xreg));
32     dec3to8 decY (.W(Y), .En(1'b1), .Y(Yreg));
33
34     regn reg_R0 (.R(BusWires), .Rin(Rin_HotOne[0]), .Clock(Clock), .Q(R0_Q));
35     regn reg_R1 (.R(BusWires), .Rin(Rin_HotOne[1]), .Clock(Clock), .Q(R1_Q));
36     regn reg_R2 (.R(BusWires), .Rin(Rin_HotOne[2]), .Clock(Clock), .Q(R2_Q));
37     regn reg_R3 (.R(BusWires), .Rin(Rin_HotOne[3]), .Clock(Clock), .Q(R3_Q));
38     regn reg_R4 (.R(BusWires), .Rin(Rin_HotOne[4]), .Clock(Clock), .Q(R4_Q));
39     regn reg_R5 (.R(BusWires), .Rin(Rin_HotOne[5]), .Clock(Clock), .Q(R5_Q));
40     regn reg_R6 (.R(BusWires), .Rin(Rin_HotOne[6]), .Clock(Clock), .Q(R6_Q));
41     regn reg_R7 (.R(BusWires), .Rin(Rin_HotOne[7]), .Clock(Clock), .Q(R7_Q));
42
43     regn reg_A (.R(BusWires), .Rin(Ain), .Clock(Clock), .Q(A_Q));
44     regn reg_IR (.R(DIN), .Rin(IRin), .Clock(Clock), .Q(IR_Q));
45     regn reg_G (.R(Alu_Result), .Rin(Gin), .Clock(Clock), .Q(G_Q));
46
47     alu U_ALU (.A(A_Q), .B(BusWires), .Sub(AddSub), .S(Alu_Result));
48
49     always_comb begin
50         if (DINout) BusWires = DIN;
51         else if (Gout) BusWires = G_Q;
52         else if (Rout_HotOne[0]) BusWires = R0_Q;
53         else if (Rout_HotOne[1]) BusWires = R1_Q;
54         else if (Rout_HotOne[2]) BusWires = R2_Q;
55         else if (Rout_HotOne[3]) BusWires = R3_Q;
56         else if (Rout_HotOne[4]) BusWires = R4_Q;
57         else if (Rout_HotOne[5]) BusWires = R5_Q;
58         else if (Rout_HotOne[6]) BusWires = R6_Q;
59         else if (Rout_HotOne[7]) BusWires = R7_Q;
60         else BusWires = 9'b0;
61     end
62
63     always_ff @(posedge Clock or negedge Resetn) begin
```

```
64     if (!Resetn) Tstep_Q <= T0;
65     else          Tstep_Q <= Tstep_D;
66 end
67
68 always_comb begin
69     case (Tstep_Q)
70         T0: begin
71             if (!Run) Tstep_D = T0;
72             else      Tstep_D = T1;
73         end
74         T1: begin
75             if (Done) Tstep_D = T0;
76             else      Tstep_D = T2;
77         end
78         T2: begin
79             Tstep_D = T3;
80         end
81         T3: begin
82             Tstep_D = T0;
83         end
84         default: Tstep_D = T0;
85     endcase
86 end
87
88 always_comb begin
89     Done = 0; IRin = 0; DINout = 0; Ain = 0; Gin = 0; Gout = 0; AddSub = 0;
90     Rin_HotOne = 8'b0; Rout_HotOne = 8'b0;
91
92     case (Tstep_Q)
93         T0: begin
94             IRin = 1;
95             DINout = 1;
96         end
97
98         T1: begin
99             case (I)
100                 3'b000: begin
101                     Rout_HotOne = Yreg;
102                     Rin_HotOne = Xreg;
103                     Done = 1;
104                 end
105                 3'b001: begin
106                     DINout = 1;
107                     Rin_HotOne = Xreg;
108                     Done = 1;
109                 end
110                 3'b010, 3'b011: begin
111                     Rout_HotOne = Xreg;
112                     Ain = 1;
113                 end
114             endcase
115         end
116
117         T2: begin
118             case (I)
```



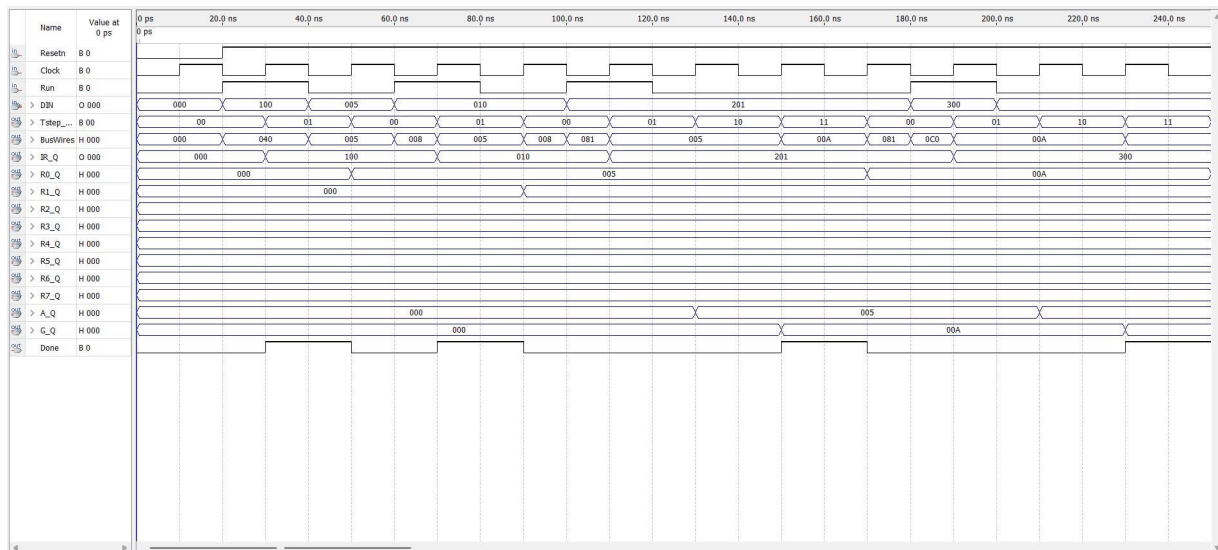
```

119         3'b010: begin
120             Rout_HotOne = Yreg;
121             Gin = 1;
122         end
123         3'b011: begin
124             Rout_HotOne = Yreg;
125             Gin = 1;
126             AddSub = 1;
127         end
128     endcase
129 end
130
131 T3: begin
132     case (I)
133         3'b010, 3'b011: begin
134             Gout = 1;
135             Rin_HotOne = Xreg;
136             Done = 1;
137         end
138     endcase
139 end
140 endcase
141 end
142 endmodule

```

Listing 4: Module simpleCPU (Processor Core)

2.1.2 Mô phỏng sóng



Hình 2.1.1: Mô phỏng Thí nghiệm 1

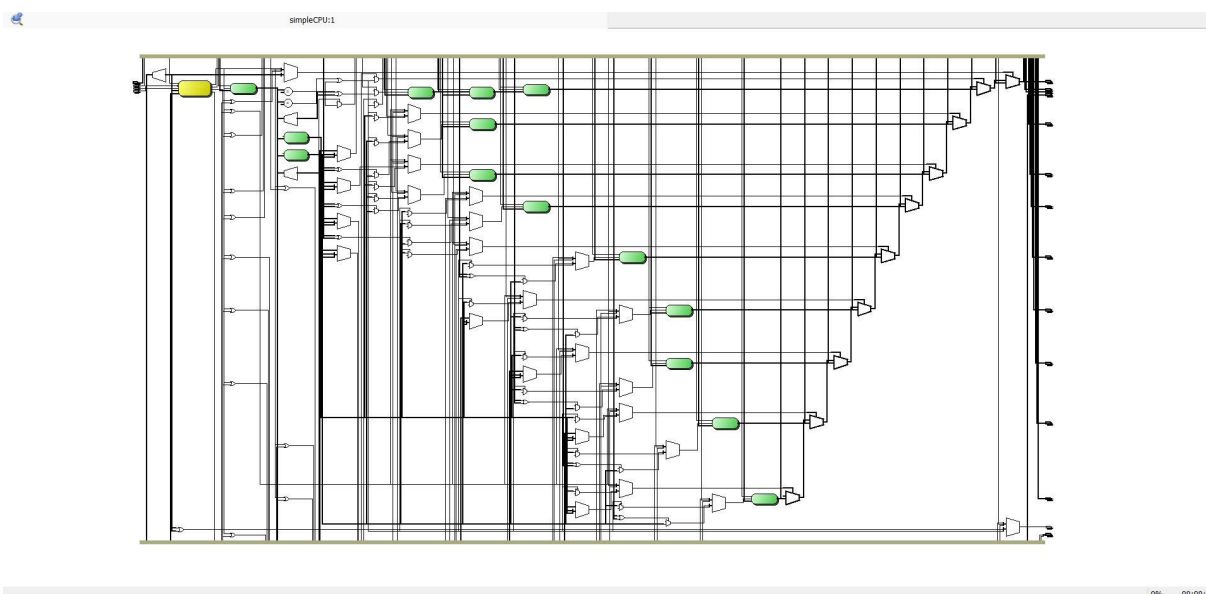
Nhận xét kết quả mô phỏng Dựa trên giản đồ sóng thu được, ta có các nhận xét chi tiết tại từng thời điểm như sau:

- Tại thời điểm $t = 20\text{ ns}$: Tín hiệu DIN được cấp giá trị $100_{(8)}$ (Octal), tương ứng với mã lệnh

001_000_000 (lệnh mvi R0). Lúc này, trên BusWires hiển thị giá trị tương ứng là 040₍₁₆₎ (Hex). Tại cạnh lên xung nhịp tiếp theo, mã lệnh này được chốt vào thanh ghi lệnh IR.

- **Tại thời điểm $t = 40\text{ ns}$:** Tín hiệu DIN được cấp giá trị tức thời là 005. Bộ xử lý chuyển sang trạng thái T1 để thực hiện nạp giá trị 005₍₈₎ vào thanh ghi R0. Khi tín hiệu Done lên mức cao (1), thanh ghi R0 đã lưu giá trị chính xác là 005₍₈₎.
- **Tại thời điểm $t = 60\text{ ns}$:** Tín hiệu DIN được cấp giá trị 010₍₈₎, tương ứng với mã lệnh 000_001_000 (lệnh mv R1, R0). Bộ xử lý thực hiện đưa dữ liệu từ R0 (giá trị 005) lên BusWires để nạp vào thanh ghi R1.
- **Tại thời điểm $t = 100\text{ ns}$:** Tín hiệu DIN mang giá trị 201₍₈₎, ứng với mã lệnh 010_000_001 (tức lệnh add R0, R1). Lúc này, cả hai thanh ghi đều đang chứa giá trị 005₍₈₎. Trên hệ thống Bus hiển thị giá trị 081₍₁₆₎ (tương ứng với mã lệnh từ DIN đang được nạp vào).
- **Về tín hiệu điều khiển:** Khi hoàn thành mỗi lệnh, tín hiệu Done luôn hoạt động đúng nhịp và được đưa lên mức cao để báo hiệu cho hệ thống sẵn sàng nhận lệnh tiếp theo.

2.1.3 RTL Viewer



Hình 2.1.2: Sơ đồ RTL Viewer của Thí nghiệm 1

2.2 Thí nghiệm 2: Processor với ROM và Counter

2.2.1 Counter và đọc file mif (memory.sv)

```

1 module counter (
2     input logic Clock,
3     input logic Resetn,
4     input logic En,
5     output logic [4:0] Q
6 );
7     always_ff @(posedge Clock or negedge Resetn) begin
8         if (!Resetn)

```

```
9      Q <= 5'b0;
10     else if (En)
11       Q <= Q + 1'b1;
12   end
13 endmodule
14
15 module MyROM
16 #(parameter int unsigned width = 9,
17   parameter int unsigned depth = 32,
18   parameter intFile = "my_ROM.mif",
19   parameter int unsigned addrBits = 5)
20 (
21   input logic CLK,
22   input logic [addrBits-1:0] ADDRESS,
23   output logic [width-1:0] DATAOUT
24 );
25
26   (* ram_init_file = intFile *) logic [width-1:0] rom [0:depth-1];
27
28   assign DATAOUT = rom[ADDRESS];
29 endmodule
```

Listing 5: Module counter và Module MyROM (memory.sv)

```
1 WIDTH=9;
2 DEPTH=32;
3
4 ADDRESS_RADIX=DEC;
5 DATA_RADIX=BIN;
6
7 CONTENT BEGIN
8   0 : 001000000;
9   1 : 011111111;
10  2 : 001001000;
11  3 : 000000111;
12  4 : 001010000;
13  5 : 000000101;
14  6 : 010000010;
15  7 : 011000001;
16  8 : 000011000;
17  [9..31] : 000000000;
18 END;
```

Listing 6: Bộ nhớ ROM 32x9 (myROM.mif)

2.2.2 Processor cập nhật cho Thí nghiệm 2 (proc.sv)

Đây là phiên bản gọn hơn của Processor được sử dụng trong Thí nghiệm 2.

```
1 module proc (
2   input logic [8:0] DIN,
3   input logic Resetn, Clock, Run,
4   output logic Done,
5   output logic PCstep,
6   output logic [8:0] BusWires,
```

```
7   output logic [8:0] R0, R1, R2, R3, R4, R5, R6, R7,
8   output logic [8:0] A, G, IR
9 );
10
11 // FSM STATES
12 typedef enum logic [2:0] {T0, T1, T2, T3, T_DATA} state_type;
13 state_type Tstep_Q, Tstep_D;
14
15 logic [8:0] AddSubResult;
16 logic [7:0] Rin, Rout;
17 logic IRin, Ain, Gin, Gout, DINout, AddSub;
18 logic [7:0] Xreg, Yreg;
19 logic [2:0] I;
20
21 assign I = IR[8:6];
22 dec3to8 decX (IR[5:3], 1'b1, Xreg);
23 dec3to8 decY (IR[2:0], 1'b1, Yreg);
24
25 // 3. FSM SEQUENTIAL
26 always_ff @(posedge Clock or negedge Resetn) begin
27     if (!Resetn) Tstep_Q <= T0;
28     else          Tstep_Q <= Tstep_D;
29 end
30
31 // 4. FSM NEXT STATE LOGIC
32 always_comb begin
33     case (Tstep_Q)
34         T0:      Tstep_D = (!Run) ? T0 : T1;
35         T1: begin
36             case (I)
37                 3'b001: Tstep_D = T_DATA;
38                 3'b000: Tstep_D = T0;
39                 default: Tstep_D = T2;
40             endcase
41         end
42         T2:      Tstep_D = T3;
43         T3:      Tstep_D = T0;
44         T_DATA:  Tstep_D = T0;
45         default: Tstep_D = T0;
46     endcase
47 end
48
49 always_comb begin
50     Rin = 0; Rout = 0; IRin = 0; Ain = 0; Gin = 0;
51     Gout = 0; DINout = 0; AddSub = 0; Done = 0;
52     PCstep = 1'b0;
53     case (Tstep_Q)
54         T0: begin
55             IRin = 1'b1;
56             PCstep = 1'b1;
57         end
58         T1: begin
59             case (I)
60                 3'b000: begin Rout = Yreg; Rin = Xreg; Done = 1; end
61                 3'b010: begin Rout = Xreg; Ain = 1; end
```

```
62         3'b011: begin Rout = Xreg; Ain = 1; end
63     endcase
64 end
65 T2: begin
66     case (I)
67         3'b010: begin Rout = Yreg; Gin = 1; end
68         3'b011: begin Rout = Yreg; Gin = 1; AddSub = 1; end
69     endcase
70 end
71 T3: begin
72     Gout = 1; Rin = Xreg; Done = 1;
73 end
74 T_DATA: begin
75     DINout = 1; Rin = Xreg; Done = 1;
76     PCstep = 1'b1;
77 end
78 endcase
79 end
80 // DATAPATH
81 regn reg_0 (BusWires, Rin[0], Clock, R0);
82 regn reg_1 (BusWires, Rin[1], Clock, R1);
83 regn reg_2 (BusWires, Rin[2], Clock, R2);
84 regn reg_3 (BusWires, Rin[3], Clock, R3);
85 regn reg_4 (BusWires, Rin[4], Clock, R4);
86 regn reg_5 (BusWires, Rin[5], Clock, R5);
87 regn reg_6 (BusWires, Rin[6], Clock, R6);
88 regn reg_7 (BusWires, Rin[7], Clock, R7);
89 regn reg_A (BusWires, Ain, Clock, A);
90 regn reg_G (AddSubResult, Gin, Clock, G);
91 regn reg_IR(DIN, IRin, Clock, IR);
92
93 always_comb begin
94     if (!AddSub) AddSubResult = A + BusWires;
95     else AddSubResult = A - BusWires;
96 end
97
98 always_comb begin
99     BusWires = 9'b0;
100     if (Rout[0]) BusWires = R0;
101     else if (Rout[1]) BusWires = R1;
102     else if (Rout[2]) BusWires = R2;
103     else if (Rout[3]) BusWires = R3;
104     else if (Rout[4]) BusWires = R4;
105     else if (Rout[5]) BusWires = R5;
106     else if (Rout[6]) BusWires = R6;
107     else if (Rout[7]) BusWires = R7;
108     else if (Gout) BusWires = G;
109     else if (DINout) BusWires = DIN;
110 end
111 endmodule
```

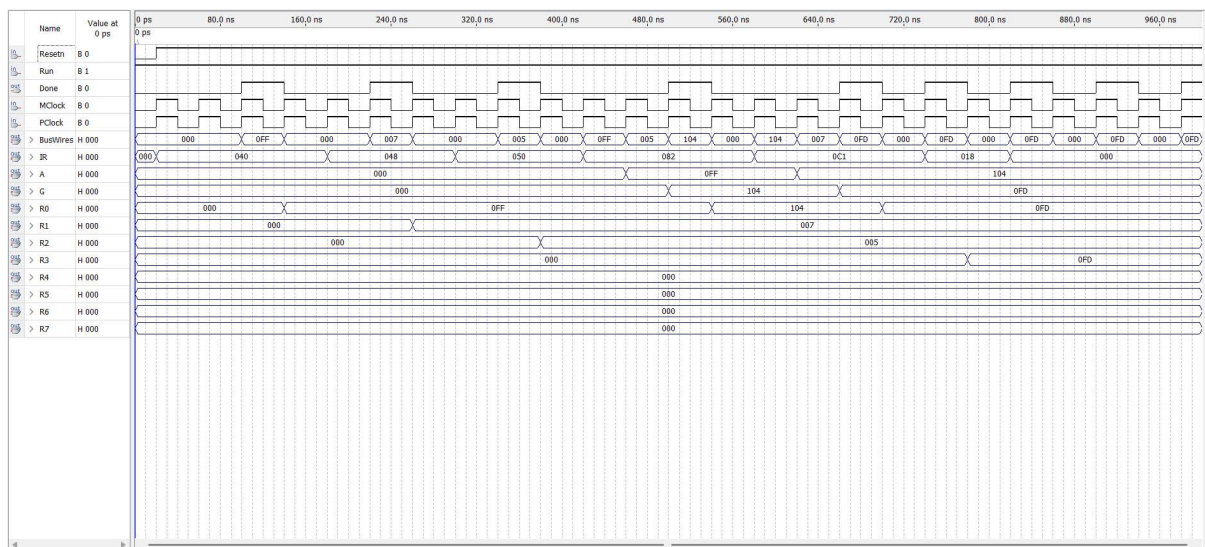
Listing 7: Processor cho thí nghiệm 2

```
1 module system_top (
2     input logic MClock, PClock, Resetn, Run,
```

```
3    output logic Done,
4    output logic [8:0] BusWires,
5    output logic [8:0] R0, R1, R2, R3, R4, R5, R6, R7,
6    output logic [8:0] A, G, IR
7 );
8
9    logic [4:0] address_wire;
10   logic [8:0] instruction_wire;
11   logic PCstep_wire;
12
13   counter U_Counter (
14       .Clock(MClock),
15       .Resetn(Resetn),
16       .En(PCstep_wire),
17       .Q(address_wire)
18   );
19
20   MyROM U_ROM (
21       .CLK(MClock),
22       .ADDRESS(address_wire),
23       .DATAOUT(instruction_wire)
24   );
25
26   proc U_Processor (
27       .DIN(instruction_wire),
28       .Resetn(Resetn),
29       .Clock(PClock),
30       .Run(Run),
31       .Done(Done),
32       .PCstep(PCstep_wire),
33       .BusWires(BusWires),
34
35       .R0(R0), .R1(R1), .R2(R2), .R3(R3),
36       .R4(R4), .R5(R5), .R6(R6), .R7(R7),
37       .A(A), .G(G), .IR(IR)
38   );
39
40 endmodule
```

Listing 8: Top Level (system_top.sv)

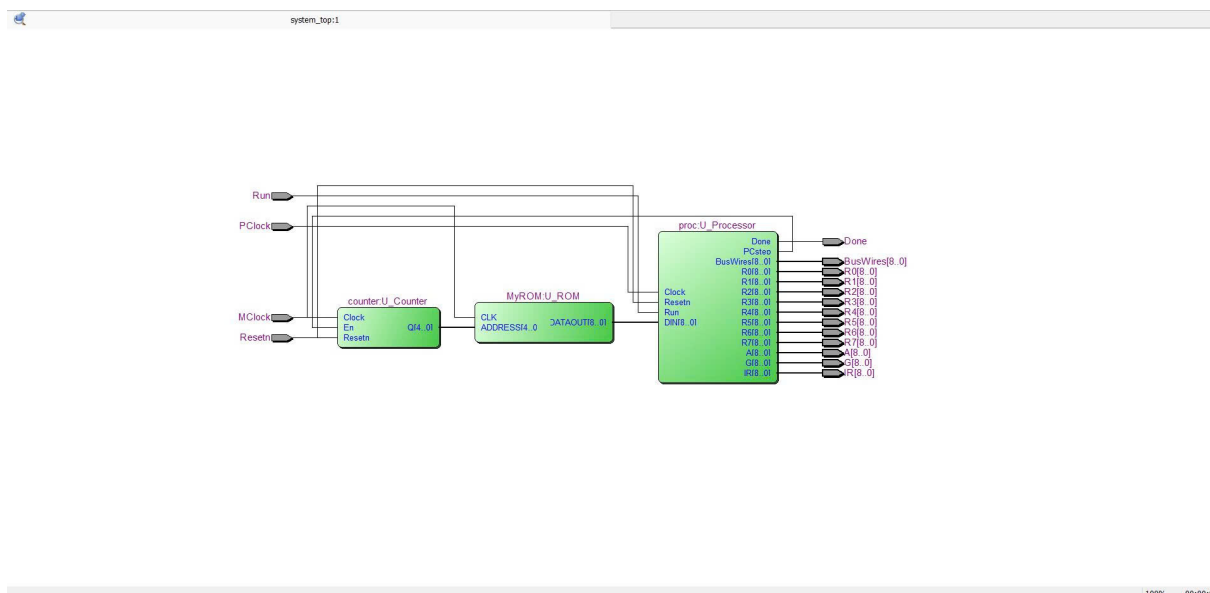
2.2.3 Mô phỏng sóng



Hình 2.2.1: *Mô phỏng Thí nghiệm 2*

Trong thí nghiệm này, Processor hoạt động tự động nhờ vào chuỗi lệnh được nạp sẵn trong ROM và địa chỉ được điều khiển bởi Counter.

2.2.4 RTL Viewer



Hình 2.2.2: Sơ đồ RTL Viewer của Thí nghiệm 2

3 AN ENHANCED PROCESSOR

3.1 Yêu cầu bài toán

Xây dựng một bộ xử lý nâng cao trên FPGA bằng cách tích hợp bộ vi xử lý đơn giản từ Lab 4 với các thành phần ngoại vi. Cần thiết kế mạch giải mã địa chỉ để phân chia vùng nhớ, cho phép CPU giao tiếp đồng bộ với bộ nhớ RAM (để đọc lệnh và dữ liệu) và các thanh ghi đầu ra (để hiển thị kết quả lên hệ thống đèn LED). Mục tiêu cuối cùng là thực thi một chương trình mã máy được nạp sẵn trong file .mif, thực hiện các phép tính toán học và điều khiển thiết bị ngoại vi theo đúng lộ trình trạng thái (FSM) và cấu trúc đường dữ liệu (Datapath) đã thiết kế.

3.2 Thiết kế chi tiết

Hệ thống bao gồm các khối chính:

- **Control Unit (proc.sv):** Điều khiển luồng dữ liệu thông qua máy trạng thái (FSM).
- **Datapath (datapath.sv):** Thực hiện các phép toán số học và lưu trữ dữ liệu trong các thanh ghi.
- **Memory (memory.sv):** Lưu trữ mã lệnh và dữ liệu.
- **System Top (system_top.sv):** Kết nối các thành phần và giải mã địa chỉ cho thiết bị ngoại vi (LED).

Khi tín hiệu `run = 1`, bộ xử lý sẽ tự động đọc lệnh, giải mã và thực thi các lệnh như: nạp dữ liệu (`mvi`), di chuyển dữ liệu (`mv`), cộng (`add`), trừ (`sub`), ghi bộ nhớ (`st`) và rẽ nhánh có điều kiện (`mvnz`).

3.2.1 Các module chức năng (datapath.sv)

```
1 module regn #(parameter n = 9) (  
2     input logic [n-1:0] R, input logic Rin, clk, output logic [n-1:0] Q  
3 );  
4     always_ff @(posedge clk) if (Rin) Q <= R;  
5 endmodule  
6  
7 module decoder3to8 (  
8     input logic [2:0] in,  
9     output logic [7:0] out  
10 );  
11     always_comb begin  
12         case (in)  
13             3'b000: out = 8'b00000001;  
14             3'b001: out = 8'b00000010;  
15             3'b010: out = 8'b00000100;  
16             3'b011: out = 8'b00001000;  
17             3'b100: out = 8'b00010000;  
18             3'b101: out = 8'b00100000;  
19             3'b110: out = 8'b01000000;  
20             3'b111: out = 8'b10000000;  
21             default: out = 8'b00000000;  
22         endcase  
23     end  
24 endmodule
```


Listing 9: File datapath.sv

3.2.2 module Datapath (datapath.sv)

Datapath là nơi thực thi toàn bộ luồng dữ liệu của CPU. Dựa trên thiết kế, nó bao gồm các thành phần chính sau:

- **Các thanh ghi R0 → R6:** Lưu trữ dữ liệu tạm thời.
- **Thanh ghi R7 (PC):** Đóng vai trò là Program Counter, có logic tự tăng hoặc nạp giá trị nhảy.
- **Thanh ghi A và G:** A giữ toán hạng đầu vào cho ALU, G giữ kết quả đầu ra.
- **Thanh ghi IR:** Lưu trữ lệnh đang thực thi.
- **Thanh ghi ADDR, D_OUT và W:** Phục vụ quá trình truy cập và ghi bộ nhớ.
- **Bộ cộng/trừ (ALU):** Thực hiện phép tính $A + Bus$ hoặc $A - Bus$ tùy thuộc tín hiệu Addsub.
- **Bộ chọn (Multiplexer):** Điều khiển việc chọn nguồn dữ liệu để đưa lên Bus chung.

```
1 module datapath (  
2     input  logic      clk, rst,  
3     input  logic [8:0] DIN,  
4     input  logic [7:0] R_in, R_out,  
5     input  logic      Ain, Gin, IRin, Gout, DINout, Addsub,  
6     input  logic      counter_en, counter_load, counter_out,  
7     input  logic      ADDR_in, D_OUT_in, W_in,  
8     input  logic      R_x_out, R_y_out,  
9     output logic [8:0] IRout, Bus, ADDR, D_OUT,  
10    output logic      W,  
11    output logic [8:0] R0, R1, R2, R3, R4, R5, R6, R7, G  
12 );  
13     logic [8:0] A;  
14     logic [8:0] alu_out;  
15  
16     regn r0(Bus, R_in[0], clk, R0);  
17     regn r1(Bus, R_in[1], clk, R1);  
18     regn r2(Bus, R_in[2], clk, R2);  
19     regn r3(Bus, R_in[3], clk, R3);  
20     regn r4(Bus, R_in[4], clk, R4);  
21     regn r5(Bus, R_in[5], clk, R5);  
22     regn r6(Bus, R_in[6], clk, R6);  
23  
24     // PROGRAM COUNTER (PC-R7)  
25     logic [8:0] PC_next;  
26     assign PC_next = R7 + 1;  
27     always_ff @(posedge clk) begin  
28         if (!rst) begin  
29             R7 <= 0;  
30         end else if (R_in[7] | counter_load) begin  
31             R7 <= Bus;  
32         end else if (counter_en) begin  
33             R7 <= PC_next;  
34         end  
35     end
```

```
35     end
36
37     // SPECIAL REGISTERS
38     regn regA(Bus, Ain, clk, A);
39     regn regIR(DIN, IRin, clk, IRout);
40     regn regADDR(Bus, ADDR_in, clk, ADDR);
41     regn regDOUT(Bus, D_OUT_in, clk, D_OUT);
42
43     // W REGISTER
44     always_ff @(posedge clk or negedge rst) begin
45         if (!rst) W <= 0;
46         else      W <= W_in;
47     end
48
49     // ALU
50     assign alu_out = Addsub ? (A - Bus) : (A + Bus);
51     regn regG(alu_out, Gin, clk, G);
52
53     // MULTIPLEXER
54     always_comb begin
55         if      (R_out[0]) Bus = R0;
56         else if (R_out[1]) Bus = R1;
57         else if (R_out[2]) Bus = R2;
58         else if (R_out[3]) Bus = R3;
59         else if (R_out[4]) Bus = R4;
60         else if (R_out[5]) Bus = R5;
61         else if (R_out[6]) Bus = R6;
62         else if (R_out[7] | counter_out) Bus = R7;
63         else if (Gout)      Bus = G;
64         else if (DINout)    Bus = DIN;
65         else if (R_x_out) begin
66             case (IRout[5:3])
67                 3'd0: Bus = R0; 3'd1: Bus = R1; 3'd2: Bus = R2; 3'd3: Bus = R3;
68                 3'd4: Bus = R4; 3'd5: Bus = R5; 3'd6: Bus = R6; 3'd7: Bus = R7;
69             endcase
70         end
71         else if (R_y_out) begin
72             case (IRout[2:0])
73                 3'd0: Bus = R0; 3'd1: Bus = R1; 3'd2: Bus = R2; 3'd3: Bus = R3;
74                 3'd4: Bus = R4; 3'd5: Bus = R5; 3'd6: Bus = R6; 3'd7: Bus = R7;
75             endcase
76         end
77         else Bus = 9'b0;
78     end
79 endmodule
```

Listing 10: File datapath.sv

3.2.3 Module proc (proc.sv) - Control Unit

Module proc đóng vai trò là đơn vị điều khiển trung tâm, sử dụng máy trạng thái hữu hạn (FSM) gồm 5 trạng thái (T0 → T4) để điều phối toàn bộ hoạt động của CPU.

Quy trình xử lý lệnh được tóm tắt như sau:

- **T0 – T1 (Fetch):** Đưa PC ra Bus để nạp lệnh từ bộ nhớ vào thanh ghi IR và tăng PC.
- **T2 (Decode):** Giải mã 3 bit đầu (IRout[8:6]) để xác định loại lệnh và thực thi các lệnh đơn (như mv).
- **T3 – T4 (Execute & Writeback):** Thực hiện các thao tác cần truy cập bộ nhớ lần 2, tính toán ALU và ghi kết quả ngược lại vào thanh ghi đích.

Giải mã lệnh (Instruction Decoding): Control Unit giải mã lệnh dựa trên IRout:

- Opcode: IRout[8:6]
- Register X (Đích): IRout[5:3] → giải mã thành Xreg (one-hot).
- Register Y (Nguồn): IRout[2:0] → giải mã thành Yreg (one-hot).

```
1 module proc (  
2     input  logic      clk, rst, run,  
3     input  logic [8:0] IRout,  
4     input  logic [8:0] G,  
5     output logic      done,  
6     output logic [7:0] R_in, R_out,  
7     output logic      Ain, Gin, IRin, Gout, DINout, Addsub,  
8     output logic      counter_en, counter_load, counter_out,  
9     output logic      R_x_out, R_y_out,  
10    output logic      ADDR_in, D_OUT_in, W_in  
11 );  
12 // Define FSM states  
13 typedef enum logic [2:0] {T0, T1, T2, T3, T4} state_t;  
14 state_t state, next_state;  
15  
16 logic [2:0] I;  
17 logic [7:0] Xreg, Yreg;  
18  
19 assign I = IRout[8:6];  
20 decoder3to8 decX (.in(IRout[5:3]), .out(Xreg));  
21 decoder3to8 decY (.in(IRout[2:0]), .out(Yreg));  
22  
23 // State Transition  
24 always_ff @(posedge clk or negedge rst) begin  
25     if (!rst) state <= T0;  
26     else      state <= next_state;  
27 end  
28  
29 // // State transition logic  
30 always_comb begin  
31     case (state)  
32         T0: next_state = run ? T1 : T0;  
33         T1: next_state = T2;  
34         T2: begin  
35             if (I == 3'b000 || I == 3'b110) next_state = T0;  
36             else next_state = T3;  
37         end  
38         T3: begin  
39             if (I == 3'b101) next_state = T0;  
40             else next_state = T4;  
41         end  
42     endcase  
43 end
```

```
42     T4: next_state = T0;
43     default: next_state = T0;
44 endcase
45 end
46
47 // FSM Output Logic
48 always_comb begin
49     R_in=0; R_out=0; IRin=0; Ain=0; Gin=0; Gout=0; DINout=0; Addsub=0;
50     counter_en=0; counter_load=0; counter_out=0;
51     ADDR_in=0; D_OUT_in=0; W_in=0; R_x_out=0; R_y_out=0; done=0;
52
53     case (state)
54     T0: begin
55         counter_out = 1;
56         ADDR_in = 1;
57     end
58     T1: begin
59         IRin = 1;
60         counter_en = 1;
61     end
62     T2: case (I)
63         3'b000: begin
64             R_out = Yreg; R_in = Xreg; done = 1;
65         end
66         3'b001: begin
67             counter_out = 1; ADDR_in = 1; counter_en = 1;
68         end
69         3'b010: begin
70             R_out = Xreg; Ain = 1;
71         end
72         3'b011: begin
73             R_out = Xreg; Ain = 1;
74         end
75         3'b100: begin
76             R_y_out = 1; ADDR_in = 1;
77         end
78         3'b101: begin
79             R_y_out = 1; ADDR_in = 1;
80         end
81         3'b110: begin
82             if (G != 0) begin
83                 R_out = Yreg; R_in = Xreg;
84             end
85             done = 1;
86         end
87     endcase
88     T3: case (I)
89         3'b001: begin
90             DINout = 1; R_in = Xreg; done = 1;
91         end
92         3'b010: begin
93             R_out = Yreg; Gin = 1;
94         end
95         3'b011: begin
96             R_out = Yreg; Gin = 1; Addsub = 1;
```

```
97         end
98         3'b100: begin
99             DINout = 1; R_in = Xreg; done = 1;
100        end
101        3'b101: begin
102            R_out = Xreg; D_OUT_in = 1; W_in = 1; done = 1;
103        end
104    endcase
105    T4: case (I)
106        3'b010, 3'b011: begin
107            Gout = 1; R_in = Xreg; done = 1;
108        end
109    endcase
110 endcase
111 end
112 endmodule
```

Listing 11: File proc.sv

3.2.4 Module Memory và Bộ nhớ

```
1 module memory #(parameter intFile = "lab5.mif")(
2     input  logic      clk,
3     input  logic [8:0] ADDR,
4     input  logic [8:0] D_OUT,
5     input  logic      wr_en,
6     output logic [8:0] DIN
7 );
8     (* ram_init_file = intFile *) logic [8:0] ram [0:127];
9
10    always_ff @(posedge clk) begin
11        if (wr_en) ram[ADDR[6:0]] <= D_OUT;
12    end
13
14    assign DIN = ram[ADDR[6:0]];
15 endmodule
```

Listing 12: File memory.sv

Bộ nhớ có kích thước 128 từ (depth=128), độ rộng 9-bit, được khởi tạo từ file lab5.mif.

```
1 WIDTH=9;
2 DEPTH=128;
3 ADDRESS_RADIX=DEC;
4 DATA_RADIX=BIN;
5 CONTENT BEGIN
6     -- Khoi tao hang so va bien
7     0 : 001001000; -- mvi R1, #1
8     1 : 000000001;
9     2 : 001010000; -- mvi R2, #0
10    3 : 000000000;
11    -- Main Loop
12    4 : 001011000; -- mvi R3, #128
13    5 : 010000000;
14    6 : 101010011; -- st R2, [R3]
```

```

15 7 : 010010001; -- add R2, R1
16 -- Delay Loop
17 8 : 001011000; -- mvi R3, #Delay
18 9 : 111111111; -- Giá trị Delay
19 10 : 000101111; -- mv R5, R7
20 11 : 001100000; -- mvi R4, #Delay
21 12 : 111111111; -- Giá trị Delay
22 13 : 000000111; -- mv R0, R7
23 14 : 011100001; -- sub R4, R1
24 15 : 110111000; -- mvnz R7, R0
25 16 : 011011001; -- sub R3, R1
26 17 : 110111101; -- mvnz R7, R5
27 -- Jump
28 18 : 001111000; -- mvi R7, #4
29 19 : 000000100;
30 [20..127] : 000000000;
31 END;

```

Listing 13: File lab5.mif

3.2.5 Module System Top

Module `system_top` có nhiệm vụ kết nối Control Unit, Datapath và Memory lại thành một hệ thống hoàn chỉnh. Ngoài ra, module này còn thực hiện chức năng giải mã địa chỉ (Address Decoding) để phân biệt giữa việc truy cập bộ nhớ RAM và truy cập LED.

- **RAM Access:** Khi địa chỉ $ADDR < 128$ (bit $ADDR[7] = 0$), dữ liệu được ghi vào bộ nhớ.
- **LED Access:** Khi địa chỉ $ADDR \geq 128$ (bit $ADDR[7] = 1$), dữ liệu được ghi ra thanh ghi LED.

```

1 module system_top (
2     input  logic      clk, rst, run,
3     output logic      done,
4     output logic      W,
5     output logic [8:0] LED,
6     output logic [8:0] PC,
7     output logic [8:0] Bus
8 );
9
10 logic [8:0] ADDR, DIN, D_OUT, R7;
11 logic [8:0] IR_Internal, G_Internal;
12 logic wr_en, LED_in;
13 logic [7:0] R_in, R_out;
14 logic IRin, Ain, Gin, Gout, DINout, Addsub;
15 logic counter_en, counter_load, counter_out;
16 logic ADDR_in, D_OUT_in, W_in;
17 logic R_x_out, R_y_out;
18
19 proc c1 (
20     .clk(clk), .rst(rst), .run(run), .done(done),
21     .IRout(IR_Internal), .G(G_Internal),
22     .R_in(R_in), .R_out(R_out), .Ain(Ain), .Gin(Gin),
23     .IRin(IRin), .Gout(Gout), .DINout(DINout), .Addsub(Addsub),
24     .counter_en(counter_en), .counter_load(counter_load), .counter_out(counter_out),
25     .R_x_out(R_x_out), .R_y_out(R_y_out),

```

```

26     .ADDR_in(ADDR_in), .D_OUT_in(D_OUT_in), .W_in(W_in)
27 );
28 datapath c2 (
29     .clk(clk), .rst(rst), .DIN(DIN),
30     .R_in(R_in), .R_out(R_out), .Ain(Ain), .Gin(Gin),
31     .IRin(IRin), .Gout(Gout), .DINout(DINout), .Addsub(Addsub),
32     .counter_en(counter_en), .counter_load(counter_load), .counter_out(counter_out),
33     .ADDR_in(ADDR_in), .D_OUT_in(D_OUT_in), .W_in(W_in),
34     .R_x_out(R_x_out), .R_y_out(R_y_out),
35     .IRout(IR_Internal), .Bus(Bus), .ADDR(ADDR), .D_OUT(D_OUT), .W(W),
36     .R7(R7), .G(G_Internal),
37     .R0(), .R1(), .R2(), .R3(), .R4(), .R5(), .R6()
38 );
39
40 assign PC = R7;
41 assign wr_en = W & (~ADDR[8]) & (~ADDR[7]);
42 assign LED_in = W & (~ADDR[8]) & (ADDR[7]);
43
44 memory c5 (.clk(clk), .ADDR(ADDR), .D_OUT(D_OUT), .wr_en(wr_en), .DIN(DIN));
45 regn LED_1 (.clk(clk), .Rin(LED_in), .R(D_OUT), .Q(LED));
46
47 endmodule

```

Listing 14: File system_top.sv

3.2.6 Module Top Level (FPGA Wrapper)

Module top_lab5 là lớp trên cùng để kết nối hệ thống với các chân vật lý của Kit FPGA (Clock, Switch, Key, LED).

```

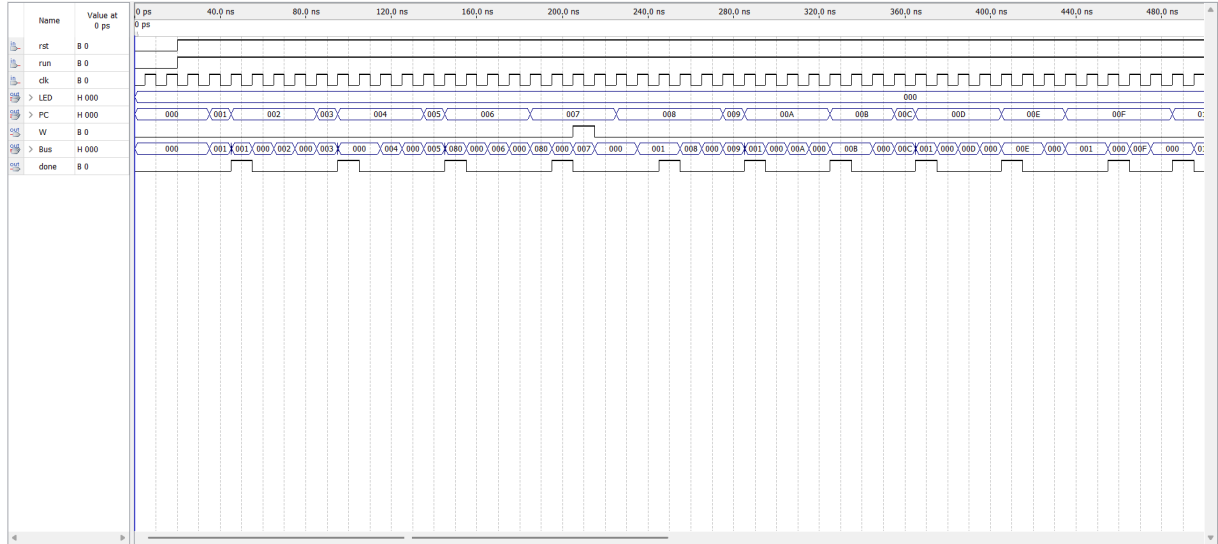
1 module top_lab5 (
2     input logic CLOCK_50,
3     input logic [17:0] SW,
4     input logic [3:0] KEY,
5     output logic [17:0] LEDR
6 );
7     // Dong bo tin hieu Run tu Switch
8     logic Run_Sync, Run_Meta;
9     always_ff @(posedge CLOCK_50) begin
10         Run_Meta <= SW[9];
11         Run_Sync <= Run_Meta;
12     end
13
14     logic [8:0] LED_Out;
15     logic Done, W;
16
17     system_top U_System (
18         .clk(CLOCK_50), .rst(KEY[0]), .run(Run_Sync),
19         .LED(LED_Out), .done(Done), .W(W)
20     );
21
22     assign LEDR[8:0] = LED_Out; // Hien thi gia tri dem
23     assign LEDR[17] = W; // Hien thi trang thai ghi
24     assign LEDR[16:9] = 8'b0;
25 endmodule

```

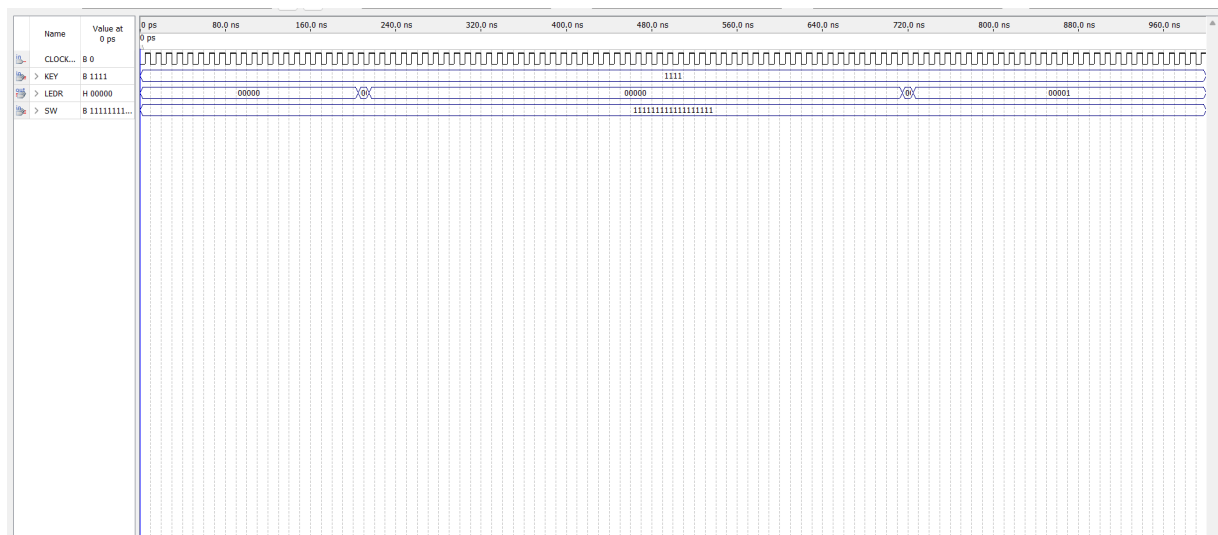
Listing 15: File top_lab5.sv

3.3 Kết quả mô phỏng

Video mô phỏng bằng Kit DE2: [Xem tại đây](#)



Hình 3.3.1: Mô phỏng system_top

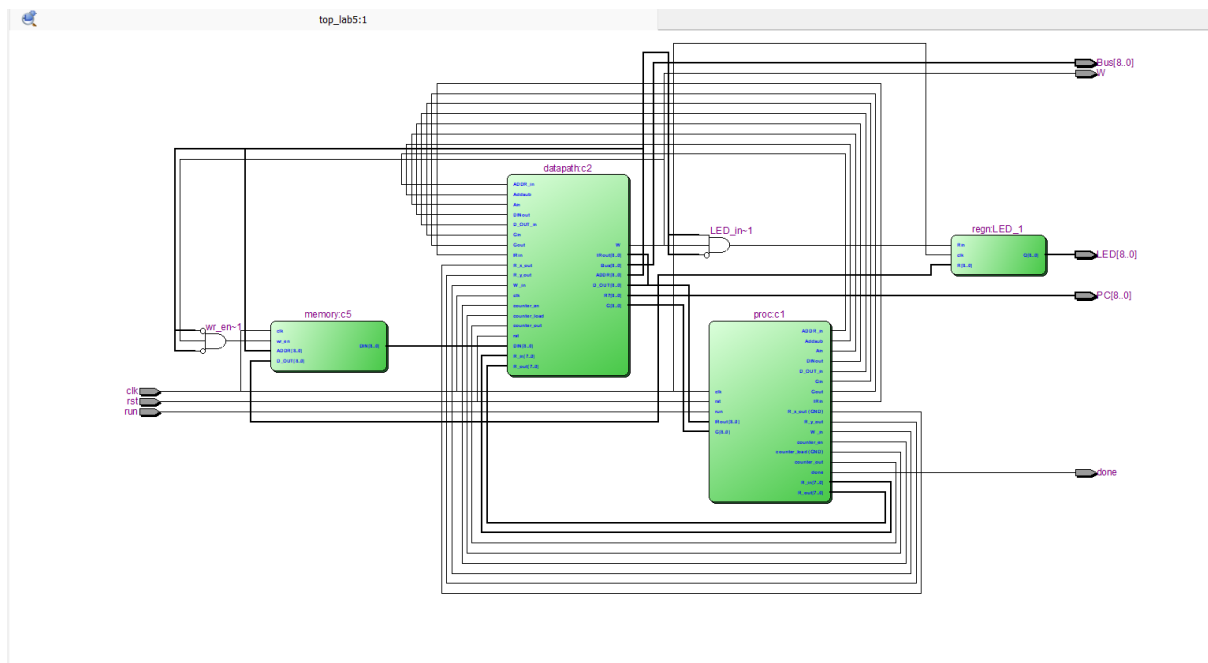


Hình 3.3.2: Mô phỏng top_lab5

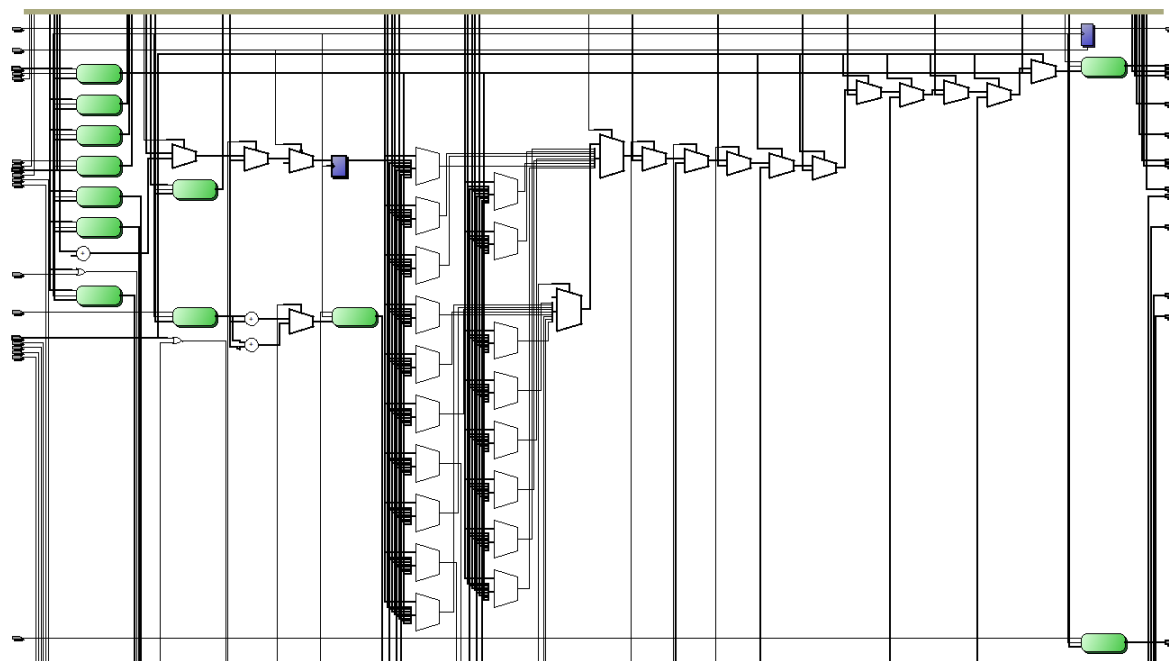
Dựa trên kết quả mô phỏng Waveform, hệ thống hoạt động đúng theo thiết kế máy trạng thái FSM. Các tín hiệu điều khiển (Control Signals) được tạo ra chính xác tại mỗi trạng thái T , đảm bảo dữ liệu được luân chuyển giữa các thanh ghi và bộ nhớ thông qua Bus chung mà không bị xung đột. Đặc biệt, quá trình giải mã địa chỉ (Address Decoding) đã phân biệt được vùng nhớ RAM và thiết bị ngoại vi LED, cho phép giá trị đếm được hiển thị đúng lúc và giữ ổn định trong suốt khoảng trễ (delay cycle)

3.4 Sơ đồ RTL Viewer

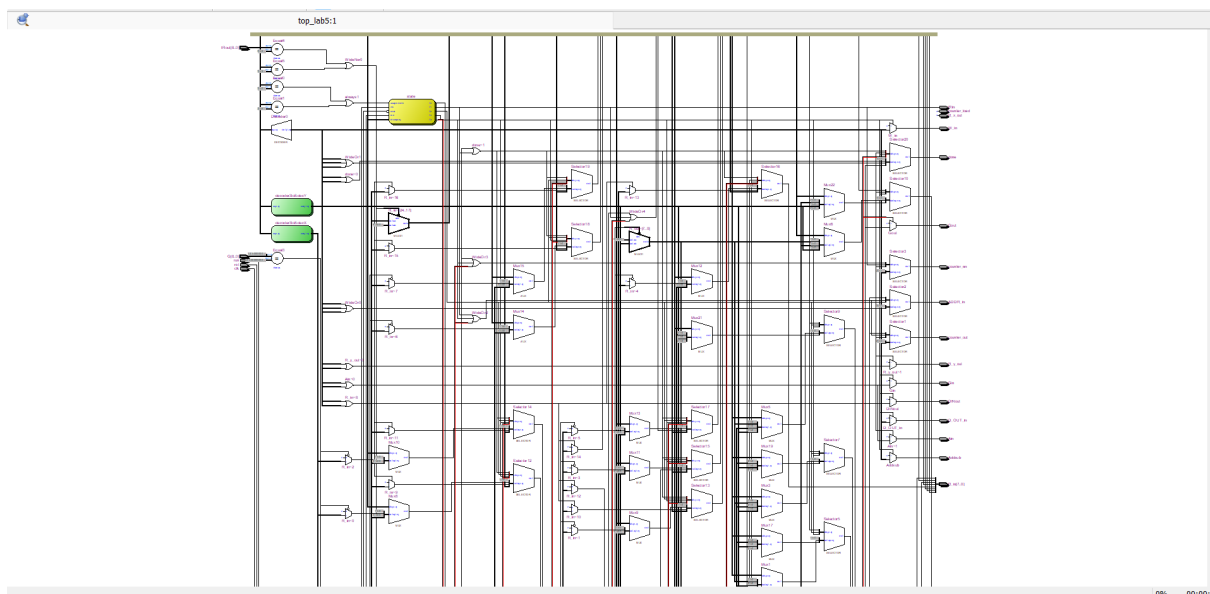
Hình ảnh dưới đây thể hiện sơ đồ mạch sau khi tổng hợp, cho thấy sự kết nối chính xác giữa các module Proc, Datapath và Memory..



Hình 3.4.1: Sơ đồ RTL Viewer của hệ thống



Hình 3.4.2: Sơ đồ RTL Viewer của datapath



Hình 3.4.3: Sơ đồ RTL Viewer của control unit (proc)

4 Lời kết

Chuỗi bài thực hành Lab 4 và Lab 5 đã cung cấp một quy trình toàn diện về thiết kế hệ thống số, đi từ các thành phần cơ bản nhất đến việc xây dựng một hệ thống máy tính hoàn chỉnh trên nền tảng FPGA.

Trong **Lab 4 - A Simple Processor**, nhóm đã đặt nền móng bằng việc thiết kế lõi xử lý trung tâm (Core Processor). Qua đó, nắm vững cơ chế hoạt động của đường dữ liệu (Datapath), cách xây dựng khối điều khiển (Control Unit) bằng FSM và quy trình thực thi lệnh cơ bản (Fetch-Decode-Execute).

Tiếp nối nền tảng đó, **Lab 5 - An Enhanced Processor** đã mở rộng hệ thống lên một tầm cao mới. Bộ vi xử lý không chỉ thực hiện tính toán nội bộ mà còn có khả năng giao tiếp với bộ nhớ ngoài (RAM) và các thiết bị ngoại vi (LEDs) thông qua cơ chế ánh xạ bộ nhớ (Memory Mapping). Việc bổ sung các lệnh phức tạp như truy xuất bộ nhớ và rẽ nhánh đã biến thiết kế thành một máy tính thực thụ có khả năng chạy các thuật toán có điều kiện.

Các kết quả đạt được sau hai bài thực hành bao gồm:

- **Thành thạo ngôn ngữ mô tả phần cứng:** Sử dụng hiệu quả SystemVerilog để mô tả các mạch tuần tự và tổ hợp phức tạp.
- **Tư duy kiến trúc máy tính:** Hiểu rõ mối tương quan giữa phần cứng (Hardware) và tập lệnh (Instruction Set Architecture).
- **Kỹ năng kiểm chứng (Verification):** Hoàn thiện kỹ năng mô phỏng (Simulation) để gỡ lỗi logic và triển khai thực tế trên kit FPGA DE-series, đảm bảo hệ thống hoạt động ổn định ở tần số thiết kế.